Integration Note for the Hardware Team

How to reach the server and what to send it

Project	RFID + Fingerprint Attendance and Access Control
Sprint / Story	Sprint 1 — US_2003 (web development side)
Author	Akshit Saini
Date	1 September 2026
Version	0.1 — draft

1. Where the server is

While we are developing, the server runs on my laptop and listens on port 3000:

```
http://<my-laptop-lan-ip>:3000
```

I will post the current IP in the group before each session — it changes when the router hands out a new lease. Both your boards and my laptop must be on the same Wi-Fi.

Important: localhost on the ESP32 means the ESP32 itself. It must be my IP address. If you cannot reach me, my firewall may be blocking port 3000 — tell me and I will open it.

2. Your device keys

Board	device_key	Door
Board 1	DEV-MAIN-8F2A41C7	Main Entrance
Board 2	DEV-LAB-5C9E30B4	Hardware Lab (restricted — card and finger)

Send it with every request. An unknown key gets a 401.

3. What to send on a scan

```
POST /api/scan
{
 "device_key": "DEV-MAIN-8F2A41C7",
 "card_uid": "04A2B7C1",
 "fingerprint_slot": 1,
 "scanned_at": "2026-09-01T09:30:44Z",
 "device_event_id": "MAIN-0011"
}
```

- card_uid: uppercase hex, no colons or spaces.
- fingerprint_slot: the R307S template ID that matched, or null if no finger was read.
- device_event_id: unique per board, and it must survive a reboot. This is what stops a duplicate when you retry.

You get back:

```
{ "allowed": true, "direction": "IN", "person_name": "Akshit Saini", "reason": "OK" }
```

Fire the relay only when allowed is true. The reason string is there for the LED and buzzer — FINGERPRINT_REQUIRED should prompt for the finger rather than just refusing.

4. When Wi-Fi is down

Buffer locally, then send up to 50 at a time to POST /api/sync with the same fields. Re-sending the same batch is safe: anything already stored comes back as a duplicate and is not counted twice.

5. Heartbeat

POST /api/heartbeat every 60 seconds with device_key and firmware_ver. The reply carries server_time, which you can use to correct clock drift.

6. Test it in two minutes

```
curl -X POST http://<my-ip>:3000/api/scan \
  -H "Content-Type: application/json" \
  -d '{"device_key":"DEV-MAIN-8F2A41C7","card_uid":"04A2B7C1","fingerprint_slot":1,"scanned_at":"2026-09-01T09:30:44Z","device_event_id":"TEST-0001"}'
```

Arduino side, the same call:

```
HTTPClient http;
http.begin("http://192.168.1.24:3000/api/scan");
http.addHeader("Content-Type", "application/json");
String body = "{\"device_key\":\"DEV-MAIN-8F2A41C7\",\"card_uid\":\"" + uid +
              "\",\"fingerprint_slot\":" + String(slot) +
              ",\"scanned_at\":\"" + isoTime + "\",\"device_event_id\":\"" + eventId + "\"}";
int code = http.POST(body);
String reply = http.getString();
http.end();
```

Use a fresh device_event_id each time, or you will get duplicate back — which is the system working correctly.

7. Things I need from you

1. Confirm the event counter survives a reboot.
2. Tell me whether you will set the clock from NTP or rely on my server_time.
3. Tell me how many events the offline buffer can hold.
4. Send me the real card UIDs once you can read them, so I can register them against people.